POMS PRODUCTION AND OPERATIONS MANAGEMENT

PRODUCTION AND OPERATIONS MANAGEMENT

Vol. 28, No. 8, August 2019, pp. 1887–1902
ISSN 1059-1478 | EISSN 1937-5956 | 19 | 2808 | 1887
DOI 10.1111/poms.13021

Implications of Application Programming Interfaces for Third-Party New App Development and Copycatting

Ling Xue
J. Mack Robinson College of Business, Georgia State University, Atlanta, Georgia 30302, USA, lxue5@gsu.edu

Peijian Song*
School of Business, Nanjing University, Nanjing 210093, China, songpeijian@nju.edu.cn

Arun Rai
J. Mack Robinson College of Business, Georgia State University, Atlanta, Georgia 30302, USA, arun.rai@eci.gsu.edu

Cheng Zhang
School of Management, Fudan University, Shanghai 200433, China, zhangche@fudan.edu.cn

Xia Zhao
Terry College of Business, University of Georgia, Athens, Georgia 30602, USA, xia.zhao@uga.edu

Digital platforms can use application programming interfaces (APIs) to support third-party development of new apps and achieve growth at an unprecedented scale. However, there is also a dilemma between original new development and copycatting by third-party suppliers. Motivated by this tension, we examined how APIs provided by digital platforms may influence two types of third-party new app development: original apps and app copycatting. We also investigated how these influences are dependent on app market conditions. We empirically tested our theoretical conjectures using data on a leading web browser platform, and applying analytics techniques on app source code to identify original apps and copycat apps. Based on a difference-in-differences identification strategy, our findings suggest that the provision of platform APIs enhance the original new development of apps. While platform APIs may facilitate app copycatting as well, our findings suggest that platform APIs can enhance app suppliers' relative attractiveness to original new development in comparison to copycatting. The enhancing effect of platform APIs on original new development is strengthened by app market potential and high market-level app complexity. The enhancing effect of platform APIs on app copycatting is strengthened by app market potential and high market concentration. Our study has important theoretical and practical implications.

Key words: new product development; platform; application programming interfaces; innovation; copycat
History: Received: April 2018; Accepted: March 2019 by Subodha Kumar, after 2 revisions.

1. Introduction

Recently, the new product development (NPD) landscape has been impacted by the development of digital platforms that have dramatically reshaped the coordination between firms and their suppliers. Instead of relying on a limited number of contracted suppliers, as in traditional supply chains, digital platform operators attract a large number of third-party suppliers to rapidly expand the core functions of the platform and achieve NPD at an unprecedented scale (Bhargava et al. 2013, Boudreau 2012, Hao et al. 2017). The focus of NPD therefore has shifted from in-house development to providing resources that support the massive contributions from third-party suppliers (Benzell et al. 2017, Billington and Davidson 2013). While this phenomenon is more prevalent in software application development in high-tech industries, other traditional industries (such as retailing and banking industries) have also been embracing digital platforms to support their NPD and operation expansion (Iyer and Subramaniam 2015, Lee and Schmidt 2017).

New product development based on digital platforms requires effective coordination and control of external third-party contributions. Digital platforms

can provide Application programming interfaces (APIs) to facilitate contributions by third-party developers, while controlling and coordinating their individual and collective contributions. APIs refer to the standardized interfaces and related development toolkits that allow third-party suppliers to access the platform's built-in software functionalities and use these functionalities in their own development (Baldwin and Woodard 2009, Tiwana 2014). Through the provision of APIs, platforms expose their core development assets to third-party suppliers and support the latter's generation of new products, often in the form of software applications (or apps) that complement and expand the fundamental platform functionalities (Benzell et al. 2017, Guha and Kumar 2018, Jacobson et al. 2012). Also, by partitioning development tasks between platforms and third-party suppliers, APIs reduce resource commitments of both parties and facilitate modular product design (Boudreau 2012, Yoo et al. 2012). These effects help enhance the overall NPD capability of the entire platform ecosystem (Gawer 2014).

When firms use digital platforms to support NPD from third-party suppliers, they also need to address imitation behavior in new app development, especially when innovation is open and intellectual property protection is not always available. While APIs facilitate new development of apps, a key ambiguity is how APIs influence third-party suppliers' orientations toward innovation specifically, do APIs provided by the platform enhance original new development of apps or app copycatting by third-party suppliers? We conjecture that APIs can have nuanced effects on original new development and copycatting behavior. From the resource deployment perspective, APIs reduce resource commitment requirements for third-party suppliers by partitioning the tasks between platforms and suppliers (Boudreau 2012, Yoo et al. 2012). As third-party suppliers are freed from developing some core functionalities on the platform, they can devote more effort to original new development (Bhargava et al. 2013). However, the modularity perspective suggests that standardized system interfaces can decrease design complexity (Schilling 2000) and make functional structures of apps more transparent to imitators (Pil and Cohen 2006). Consequently, modularization through APIs may reduce the barriers of imitation, thereby facilitating copycatting (Ethiraj et al. 2008) and eventually hurting original new development. Given the theoretical tension and the lack of empirical evidence on the effects of APIs, we theorized and empirically investigated the effects of platform APIs on both original new development of apps and app copycatting of third-party suppliers. This timely investigation can contribute to our understanding of how NPD can be fostered by platform strategies.

We also examined how the effects of APIs on original new development of apps and app copycatting are dependent on app market conditions. Such examination is mainly motivated by the literature that recognizes the role of market conditions in shaping the patterns of innovation activities (Fabrizio and Thomas 2012, Turner et al. 2010). We focused on three market conditions: (i) *market potential for apps*, captured by the size of platform user base; (ii) *concentration of app market*, captured by the market share distribution of competing apps; and (iii) *market-level app complexity*, captured by the overall coding scales of apps that may lift the barriers for innovation and imitation in app development. We consider these three specific market conditions because they reflect the two key characteristics of digital platforms: network effect and technological requirement. Market potential reflects the cross-side network effect (the influence of user-side of the platform on supplier-side development) and market concentration reflects the same-side (negative) network effect (competition among participants on the supplier-side). Market-level app complexity reflects the general technical requirements that app suppliers need to meet to achieve digital NPD.

The empirical context of our study is Mozilla Firefox, a leading web-browser platform, which serves as an open software platform attracting third-party development of "add-on" apps that run on it. We used content analysis techniques to distinguish between original new apps and copycat apps based on the app source code, and employed Firefox's release of software development kit (SDK) APIs (in December 2010) as a quasi-experimental setting to study the effects of APIs on original new app development and app copycatting. We utilized a difference-in-difference (DID) approach to examine how the release of SDK APIs generated differential effects between expanding app categories with stronger driving forces for subsequent new app releases and non-expanding app categories. We also examined how app market conditions shape the identified differential effects of APIs.

The contribution of our research to the literature is twofold. First, we found that the release of SDK APIs enhanced original new development of apps by third-party suppliers. While app copycatting increased as well after the release of SDK APIs, we found that SDK APIs also motivated third-party suppliers lean relatively more toward using original new development and less toward using app copycatting to seize market opportunities. Such effort-shifting effect mitigates the overall enhancing effect of APIs on app copycatting. These findings add to the literature by elucidating how APIs support platform-based NPD and reconcile

existing conflicting views regarding the implications of product modularity (implemented through APIs) for product imitation. Second, our results suggest that the effects of API are conditional on the app market environment. We found that the enhancing effect of platform APIs on original new development is strengthened by app market potential and market-level app complexity. The enhancing effect of platform APIs on app copycatting was strengthened by app market potential and market concentration. These insights contribute to the literature by clarifying the joint influence of the design of modular platforms through APIs and market conditions on third-party NPD and product imitation.

2. Theoretical and Hypothesis Development

2.1. Platform-Based New Product Development and Application Programming Interfaces

Platform strategy for NPD has been examined in different research disciplines and from diverse theoretical perspectives. In the operations management literature, the focus has been primarily on the various ways in which individual producers use the principles of platform to design new product families (e.g., Jiao et al. 2007, Jose and Tollenaere 2005, Krishnan and Loch 2005). Product platforms consisting of a set of components with their standardized interfaces and a modular architecture allow producers to accomplish NPD by reusing and combining the existing components through the underlying interfaces (Jiao et al. 2007, Vickery et al. 2016). Benefits of utilizing platform-based NPD include enhanced design flexibility, increased market responsiveness, and reduced development complexity and cost (Zhang 2015).

The technology management literature has focused mostly on how contemporary digital platforms shift the innovation paradigm beyond organizational boundaries (Baldwin and Woodard 2009). Since platforms provide key building blocks based on which third-party suppliers can develop complementary new products or services, digital platforms essentially form "ecosystems" of open innovation (Gawer 2014). Taking software platforms as an example, APIs define the ways for the core systems of platforms and third-party digital new products (which are typically in the form of software apps) to integrate with each other. By releasing APIs to attract a large number of third-party suppliers to develop digital new apps, platforms can achieve inter-organizational NPD collaboration beyond the scope of traditional value chain partnership.

We take a new perspective and consider APIs as a mechanism for firms to implement platform strategies to support NPD in the open innovation paradigm. First, APIs support resource deployment by dividing development tasks between platforms and third-party suppliers (Boudreau 2012, Yoo et al. 2012). NPD is essentially a process through which technological resources and capabilities of the platform can be shared with third-party suppliers, decreasing their efforts and skill requirements (Von Hippel and Katz 2002). Second, APIs implement the principles of system modularity. By clearly defining the interfaces between the platform and third-party components and specifying the interactions through these interfaces, APIs help realize the separateness principle (Schilling 2000) and the specificity principle (Ulrich 1995) in system modularity. These modularity principles enhance the coordination of NPD in the inter-organizational operational context. Third, platform APIs also serve as a control mechanism for platform operators to govern innovation ecosystems (Tiwana 2014). By managing standardized interfaces, APIs dispense with exhaustive monitoring of a large supplier base and effectively regulate suppliers' NPD behavior in a broader operational environment.

2.2. Original New Development of Apps and App Copycatting

To examine NPD in our research context of software platforms, we distinguish between *original new development of apps* and *app copycatting* by third-party app suppliers. Original new development refers to the case in which third-party suppliers create their new apps that are distinct from the existing apps in terms of code content. In contrast, app copycatting refers to the case in which the new apps created by third-party suppliers replicate the existing apps in code content. While original new app development reflects suppliers' efforts to carry out creative ideas into practice in digital NPD, app copycatting mainly captures the behavior to imitate others' NPD outcomes without making salient original inputs (Wang et al. 2018).

It is worth noticing that the distinction in code content may reflect different modalities of innovation. First, the third-party supplier may realize his/her own innovative ideas and implement creative functions in the new app. Second, the supplier might adopt the existing ideas on functions from other suppliers and implement similar functions using his/her own ways of coding. The first case captures innovation based on *what* the software accomplishes while the second case involves imitating the functions that are accomplished. As such, it captures innovation based on *how* the software accomplishes the objectives. We draw a distinction between original new development of apps and copycat apps in the current way for several reasons.

First, incremental innovation prevails in digital NPD (Bessen and Maskin 2009, Yoo et al. 2012). In this process, developers exert substantial efforts in reconstructing the existing functions in distinct ways so that the reconstructed functions can be integrated into the existing systems. This process requires considerations of interdependence and responsiveness, as well as compatibility of the reconstructed functions with the remaining system. Developers also learn through the replication of the existing functions in their own programs, which is often a necessary step toward creating new functions (Bikhchandani et al. 1998). Second, recombining the existing functions in a more efficient or creative way is an integral part of software innovation (Sanchez and Mahoney 1996). Indeed, architectural innovation, which involves the application of architectural principles to recombine functions, is a major class of product innovations (Henderson and Clark 1990), including software innovations (Lyytinen and Rose 2003). Therefore, defining original new development based only on new functions fails to capture the different types of creative efforts that are integral to software innovation. Third, from a practical perspective, in our research context of open-source app development, it is less likely for an app supplier to develop his/her original code just to copy app functions from others. The open-source app code of other apps is not protected by copyright. There is no need for imitators to hide their copycatting behavior using their own original code. Doing so increases the app supplier's effort cost without bringing extra benefit. Therefore, the original code should at least involve a certain degree of learning and innovation. We focused on the originality of code content to consider all intentions, processes, and actual outcomes related to app innovation and distinguished the original app from the most obvious copycatting behavior of directly copying the source code from others without substantial modification.

2.3. Dual Role of Modularity in Original New Development of Apps and App Copycatting

Imitation in NPD is a salient issue in both the operations management literature (Pun and DeYong 2017, Zahra and Das 1993) and the technology management literature (Ethiraj et al. 2008, Pil and Cohen 2006, Rivkin 2000). While product modularity fosters innovation activities in NPD by enhancing the transparency of product architecture, flexibility in design adjustment, and control of complexity (Sanchez and Mahoney 1996, Vickery et al. 2016), these benefits also reduce the barriers for imitation and can potentially erode the sustainability of innovation advantages (Pil and Cohen 2006). Loosely coupled modules and standardized interfaces provide followers with better capabilities to trace inventors' development paths and use mix-and-match replication approaches (Rivkin 2000). Therefore, individual inventors often face the trade-offs between simplifying product architecture through modularization to facilitate innovation and maintaining architectural complexity to deter imitation (Ethiraj et al. 2008).

The extant studies, however, have primarily addressed the issue from the perspective of individual producers and individual closed product systems (Pil and Cohen 2006). Our study, with a platform perspective, departs from the past work in two noteworthy ways. First, the platform perspective that we adopt is less concerned about individual producers; instead, it focuses on the NPD outputs of the ecosystem. Second, prior studies have assumed that individual producers have the right to control product modularity to pursue innovation or address imitation. In our setting, while APIs implement modularity principles, they are beyond the control of individual third-party suppliers and typically reside with the platform owner. Rather, third-party suppliers can reallocate their development efforts toward original new development of apps or app copycatting in response to the provision of APIs and modularity.

2.4. Hypothesis Development

Informed by modularity principles, platform APIs can bolster third-party suppliers' original new development of apps in two ways. First, app suppliers can use APIs to efficiently assemble development resources and achieve combinatorial innovation (Danese and Filippini 2013, Sanchez and Mahoney 1996). The mix-and-match approach that enables recombination of development components substantially reduces time-to-market and accelerates the introduction of new products (Ethiraj and Levinthal 2004). Second, as APIs release app suppliers from the burden of developing similar functionalities provided by the platform (Evans et al. 2006), app suppliers become empowered to pursue more original ideas. They can reallocate effort from replicating fundamental functions to designing more unique and creative app features that add value to the platform (Tiwana 2014). In practice, there is much recent anecdotal evidence on how APIs are employed as a new initiative to sprout new development by third-party suppliers. For example, Uber opened its APIs to support partners to create innovation in their own businesses, such as on-demand delivery, based on Uber technology (Domis 2016). We therefore hypothesized that

H1A. *Platform APIs exhibit an enhancing effect on third-party suppliers' original new development of apps.*

Platform APIs may also spur app copycatting, as informed by how modularity may facilitate product

imitation (Pil and Cohen 2006, Rivkin 2000). Platform APIs can facilitate the learning of imitators. The use of platform APIs makes app architecture and resource deployment more transparent to imitators. The loose-coupling between third-party app functionalities and platform functionalities also reduces challenging steps for imitators to reverse engineer and experiment (Rivkin 2000). Consequently, the lower learning barriers can make copycatting a more appealing choice for market followers, especially those with limited capabilities of the original development, to quickly seize market opportunities (Bockstedt et al. 2016). Moreover, the use of platform APIs decreases the actual costs of copycatting. The principles of modularity and standardization implemented through APIs help reduce the complexity of copying functionalities. Using the same set of APIs as original app suppliers, imitators would find it more convenient to infuse the existing code and development outcomes into their copycat apps (Bessen and Maskin 2009).

Although platform APIs may facilitate both original new development and copycatting, whether platform APIs would increase the relative attractiveness of original new development to app suppliers, in comparison to copycatting, is of theoretical importance. We proposed that platform APIs will drive app suppliers to lean more toward original new development as opposed to app copycatting. We conjecture that APIs generate an "effort-shifting" effect that makes app suppliers focus relatively more on original new development and relatively less on copycatting, despite that the overall original new development and the overall copycatting can both be increased by the use of platform APIs.

The use of APIs facilitates both original new development and copycatting by reducing technological barriers (Evans et al. 2006). This facilitating-effect should, however, be more prominent for original new development than for copycatting mainly because technological barriers for original new development are higher compared to those for copycatting. APIs can empower more suppliers who have creative ideas and certain experiences to overcome technological barriers and reallocate more of their efforts from replicative activities to original new development (Lau et al. 2011). Therefore, the use of APIs can make the option of original new development rather than copycatting more appealing to more suppliers. Nevertheless, APIs may still spur more copycatting by encouraging more inexperienced market newcomers to expand the whole market. Experienced app suppliers with innovative ideas are likely to reallocate their efforts and lean more toward using original new development rather than app copycatting to seize market opportunities. Thus, we hypothesized that

H1B. *Platform APIs increase the relative attractiveness of original new development to third-party app suppliers, in comparison to app copycatting.*

Greater market potential, reflected by the size of platform user base, can generate more tangible benefits (e.g., economic returns) and intangible benefits (e.g., reputation) to third-party developers through cross-side network effects (Chu and Manchanda 2016, Song et al. 2018). These benefits are integral to motivating NPD activities (Acemoglu and Linn 2004). In this regard, greater market potential should incentivize third-party app suppliers to engage more in new app development, through either original new development or copycatting (Fabrizio and Thomas 2012).

We expected that in the presence of greater market potential, platform APIs would be more effective in bolstering both original new development of apps and app copycatting. Through partitioning development tasks and providing resource deployment, APIs improve the efficiency of both original new development of apps and app copycatting. When there is a larger base of platform users, app suppliers (including both innovators and imitators) are in greater need of such efficiency to serve more users. They should be more prone to leveraging APIs to achieve efficient development, and consequently, platform APIs should sprout original new app development and app copycatting to a greater extent.

Additionally, with greater market potential, app suppliers (including both innovators and imitators) need more development flexibility to meet more diverse and uncertain user needs (Yoo et al. 2012). As APIs provide more development flexibility by supporting modular system designs and recombination of functionalities, app suppliers can achieve rapid trial-and-error learning and use more experimental development to serve diverse user needs (Danese and Filippini 2013). Therefore, with greater market potential, app suppliers can leverage the enhancing effects of platform APIs on new development (including both original new development and copycatting) to a greater extent. We thus hypothesized that

H2. *Market potential for apps* (*a*) *strengthens the enhancing effect of platform APIs on original new development of apps; and* (*b*) *strengthens the enhancing effect of platform APIs on app copycatting.*

The app market concentration refers to the case in which a small set of popular apps attract the majority of users (Turner et al. 2010). The existing literature has suggested the countervailing effects of market concentration on new product innovation. On the one hand, market concentration fosters innovation because market leaders can leverage their market

power to appropriate more returns from their investments in innovation (Scherer 1992). On the other hand, market concentration can reduce competitive pressure, which is an important impetus for product innovation (Curry and George 1983). Reflecting on the nuanced relationship between market concentration and innovation, prior studies have suggested that attention should be directed at examining the contingent effects of concentration in combination with other factors, such as technology resources (Turner et al. 2010).

We expect that concentration undermines the enhancing effect of platform APIs on original new development of apps for two main reasons. First, for market leaders, concentration leads to mixed incentives for original new app development, as suggested by the extant literature. In choosing their strategies for new app development, the few leading app suppliers are likely to focus more on their mutual competition and less on other broad technological opportunities. As a result, platform APIs may play a less vital role in motivating original new development in concentrated app markets. Second, for market followers, market concentration suggests that few opportunities and demands are left for them to pursue. As a result, market followers lack incentives for original new development even when they are provided with supporting mechanisms, such as platform APIs. Therefore, the bolstering effect of platform APIs on original new development of apps should become weaker when the market concentration is higher.

In contrast, market concentration can reinforce the enhancing effect of platform APIs on app copycatting. In concentrated markets, the app products of market leaders become the clear targets of copycatting for followers (Bikhchandani et al. 1998). Followers are also left with less room to compete through creative development. Consequently, followers are more likely to rely on the copycatting strategy (Lieberman and Asaba 2006). When provided with platform APIs, they are also likely to gear their development toward copycatting. Even though some market leaders' incentives for copycatting may diminish due to market concentration, the overall market is likely to see more copycatting in response to the provision of platform APIs because market followers usually account for the majority of market participants. Therefore, we hypothesized that

H3. *Concentration of app market* (*a*) *weakens the enhancing effect of platform APIs on original new development of apps; and* (*b*) *strengthens the enhancing effect of APIs on app copycatting.*

We consider the overall technological complexity of app markets as a contingency factor. We are motivated to do so as product complexity has been suggested to not only increase the requirements and barriers for NPD, but also influence the effect of modularity (Peng et al. 2014, Vickery et al. 2016). The overall technological complexity of an app market is reflected by the average complexity of apps in the market (referred to as *market-level app complexity* hereafter). Market-level app complexity is characterized by the presence of different coding elements that are integrated in ways that make it difficult for non-original developers to digest and copy the app content. Larger apps with more functionalities and coding elements are often more complex as they contain more detailed elements (or component complexity) and more complicated cross–element interdependencies (or structural complexity) (Pil and Cohen 2006). The markets for more complex apps increase the technological requirements for both original new development of apps and app copycatting. Users in these markets are likely to expect more advanced and sophisticated functionalities from new apps that challenge innovators. For imitators, the information processing load associated with reverse engineering and replicating complex apps can surpass their cogitative limits, as suggested by the bounded rationality perspective (Vickery et al. 2016).

The role of platform APIs in facilitating new app development therefore becomes more crucial in the presence of higher market-level app complexity. App suppliers face higher standards to compete in markets that require more complex apps. In this case, suppliers of original new apps should rely more on using platform APIs to complete some standard development tasks, so that they can better allocate their efforts to accomplish more complex part of original new development. Likewise, market-level app complexity also creates challenges for imitators to copy components from other complex systems. More technical sophistication would be required to incorporate the complex copied components into copycat apps. In this case, it would be important for suppliers of copycat apps to use platform APIs to support their imitation strategies. We therefore hypothesized that

H4. *Market-level app complexity* (*a*) *strengthens the enhancing effect of platform APIs on original new development of apps; and* (*b*) *strengthens the enhancing effect of platform API on app copycatting.*

3. Methodology

3.1. Research Context

Our empirical setting is a major web browser platform—Mozilla's Firefox. Firefox is the second most widely used desktop web browser (trailing Google

Chrome) (StatCounter 2017), with approximately 12% in market share. As a free web browser, Firefox relies on a platform ecosystem to extend its product boundaries by encouraging third-party developers to supply complementary "add-on" apps. These apps extend the platform's capabilities in various ways, such as privacy protection, videoconferencing, and anti-phishing filters. As of October 2016, there were more than 15,000 apps, spanning 14 categories on Firefox, provided by more than 1400 third-party developers. These 14 app categories include alerts & updates, appearance, bookmark management, download management, feeds/news/blogging, games & entertainment, language support, photos/music/videos, privacy & security, shopping, social & communication, tabs, web development, and other.

Firefox provides APIs to support third-party app development. The first set of APIs, named as XPCOM APIs, provided access to a set of core platform functionalities, for example, file and memory management, threads, basic data structures (strings, arrays, variants), to developers. On December 9, 2010, Firefox released the second set of APIs, that is, SDK APIs. Compared to XPCOM APIs, SDK APIs significantly simplify many common tasks in developing, testing, and packaging apps, and are also forward-compatible with the new multiple process architecture planned for Firefox. The user interface (UI) components available in the SDK APIs are designed to align with the usability guidelines for a better and more consistent experience on Firefox. SDK APIs further offer better security for apps not to harm users' operation systems, new restartlessness function when users update their apps, and supports on Firefox Mobile. More information about Firefox's APIs is described in online Appendix A.

We study the release of SDK APIs in our empirical research setting for several reasons. First, the platform records all third-party apps developed for Firefox since the platform was launched in 2004. For each app, the platform provides detailed historical information (e.g., initial launch time and updates). We can thus achieve a relatively complete observation of app-side new development. Second, according to the explanation from Firefox, SDK APIs were provided mainly to reduce the complexity in previous sets of APIs (XPCOM APIs) and facilitate more convenient app development. This objective is more aligned with our theoretical consideration. Third, the code of most apps is open-source and reveals details on the app design and which APIs the app invokes. The source code allows us to detect original apps and copycats. Fourth, the release of SDK APIs is largely exogenous to individual developers, and therefore minimizes endogeneity concerns. Finally, Firefox and its apps are free for users, helping avoid some confounding effects of pricing on original new development of apps and app copycatting.

3.2. Data and Measurement

The observation window is from June 2010 to June 2011, with 26 weeks before and after the official release of SDK APIs on December, 2010. The unit of analysis is the app category–week combination. A general observation of our data suggests that weekly data provides better granularity of observation on app release patterns than daily or monthly data. We collected related information about Firefox add-on apps from the official website (https://addons.mozilla.org/en-us/firefox).

3.2.1. Copycat Detection. An integral part of our analysis is to identify original new apps and copycat apps. We leverage the open-source nature of Firefox and its add-on apps and adopt analytics techniques in software engineering to conduct code content analysis. We crawled all app source code files from Firefox, with a total of 83,557 code files for 15,911 apps. We identify original new apps and copycat apps based on the similarity of code content. In software engineering, automated similarity comparison can detect text similarity and semantic (content) similarity of source code efficiently (e.g., Burrows et al. 2007), but the detection of higher-level concepts such as purposes of functionalities and system architecture is still challenging. However, text and content similarity comparison is a valid approach that has been employed in software plagiarism judgment (e.g. Bin-Habtoor and Zaher 2012). Therefore, we employ this approach for copycat detection.

The two main types of similarity detection techniques in the software engineering literature include fingerprints and content comparison (e.g., Lukashenko et al. 2007, Mozgovoy 2006). Fingerprints are descriptive statistical information about the software program, such as number of operators and operands in a code file, average number of terms per line and number of unique programming terms. Code files are considered similar if their fingerprints are close to each other based on certain similarity metrics, such as Euclidean distance (Lukashenko et al. 2007).

In contrast, content comparison techniques assess app similarity by comparing program structure. These techniques search for matching contiguous sequence or parameters of substrings within the programs (Mozgovoy 2006). To build structural information of strings for content comparison, we use latent semantic analysis (LSA), a statistical technique for textual content analysis to measure the structural similarity among source code files (Cosma and Joy 2012). This technique has been widely applied in the information systems field (e.g., Moody and Galletta 2015,

Sidorova et al. 2008). The idea of LSA is that there exists a set of latent dependencies between the words and their contexts (phrases, paragraphs and texts). LSA approximates the structure of term usage among documents through the singular value decomposition (SVD) approach and uncovers the important relations between terms by reducing the noise in the data (Kontostathis and Pottenger 2006). Furthermore, changes to the document structure do not affect the detection of similar documents because LSA treats each document as a bag of words.

The technical details of using LSA to assess code similarity are provided in online Appendix A. Code similarity values range between 0 and 1, with higher values representing greater similarity between the source code files of apps. After computing similarity values, we judge whether the new launched apps are original new apps or copycat apps using a threshold level of 0.9 (Cosma and Joy 2012, Wang et al. 2018). We also use an alternative threshold of similarity in a robustness check.

3.2.2. Variables. In our empirical analysis, original new development of apps is captured by a variable *OriginalNew* measured using the (logged) number of original new apps in each category-week. App copycatting is captured by a variable *CopycatNew* measured using the (logged) number of new copycat apps in each category-week. We log transformed these two variables to address the skewness in the distribution. The effects of APIs are examined using two dummies in a difference-in-difference setting (explained in more detail in section 3.3). A dummy *PostAPIs* takes a value of 0 for the weeks before the release of SDK APIs, and a value of 1 for the weeks after the release of SDK APIs. A dummy *Expansion* takes a value of 1 for app categories with market expansion (which is equivalent to the treatment group), and a value of 0 for non-expanding app categories (as the comparison group). Additional detail about the constructing these groups are provided in section 3.3.

The variable for market potential (*MktPotential*) is measured using the number of existing Firefox users each week (log-transformed). We log transformed the *MktPotential* value. The average weekly Firefox users were about 0.18 billion ($e^{18.961}$) over our observation window and was relatively stable in this period. The variable for market concentration (*MktConcen*) is measured using the weekly Herfindahl index of the app category. The Herfindahl index is defined as $HI = \sum_{i=1}^{N} (x_i/x)^2$, where x_i is the number of users for app i, and x is the total number of users for the corresponding app category. Therefore, x_i/x is the share of app i. A higher Herfindahl index indicates that users are concentrated in a few top apps in the category. The variable for market-level app complexity (*MktAppComplex*) is measured using the average size of code files in megabytes at the category-week level (Rai et al. 2009). A larger average size of code files indicates a higher level of app complexity.

We also use several control variables. We controlled for the general growth trend of app category by including a variable (*CatGrowth*) measured by the average growth rate of apps in each category in the past three months. App categories with higher growth rate are expected to have more original new apps and copycat apps. We also include two variables to control for app quality and user feedback. The first one is *AppRating* measured using the average star ratings (1-lowest, 5-highest) of all apps at the category-week level. The second one is the *ReviewNumber* measured using the total number of user reviews for all apps at the category-week level. App categories with higher average ratings and more views are expected to have more original new apps and copycat apps. To control for app competition, we included a variable *AppNumber*, which is measured using the weekly total number of existing apps in each category. Larger categories with more apps are expected to have more original new apps and copycat apps. To control for the impact of platform update, we included a binary indicator *PlatformUpdate*, which has a value of 1 if a platform update was released in the corresponding week, and a value of 0 otherwise. Platform updates are expected to trigger more original new development and copycatting. Finally, to control for the app review process, we included a variable *AppReviewTime*, which is measured using the average review time (in the number of days) for the new released apps in the corresponding category–week. App categories with a shorter average review time are expected to have more original new apps and copycat apps. Table 1 shows variable definition and summary statistics. The correlation matrix for these variables is shown in online Appendix B.

3.3. Quasi Experiment Design

Our empirical analysis leverages a quasi-experiment setting, which can provide greater validity in such inference than purely statistical adjustments (Shadish et al. 2002). We take the provision of SDK APIs by Firefox as an exogenous shock for the quasi experiment. When released, SDK APIs were accessible to all app categories and third-party suppliers. Therefore, our study is a setting in which the exogenous shock impacts all subjects and our inferences are based on the variation between different groups (as explained below) that subject to the impact of shock to different extents (Kearney and Levine 2015).

We used a research design where we defined the following: (i) the provisioning of the SDK APIs as the

Table 1 Constructs, Measures, and Descriptive Statistics

Variables	Measurement	Mean	SD	Min.	Max.
OriginalNew	Number of original new apps released in each category-week (log)	1.482	0.311	0.000	2.653
CopycatNew	Number of new copycat apps released in each category-week (log)	1.267	0.221	0.000	2.301
PostAPIs	Dummy (=1 after the release of SDK APIs; =0 otherwise)	0.500	0.500	0.000	1.000
Expansion	Dummy (=1 Expansion group; =0 Non-Expansion group)	0.500	0.500	0.000	1.000
MktPotential	Total number of Firefox users in each week	18.961	0.182	18.736	19.385
MktConcen	Herfindahl index of number of users in each category-week	0.405	0.212	0.071	0.770
MktAppComplex	Average size of code file for the existing apps in each category-week	0.237	0.370	0.102	2.361
CatGrowth	Average 3-month growth rate of app number in the category	0.007	0.018	0.000	0.233
UserRating	Average review ratings of apps in the category-week	4.128	0.156	3.776	4.345
ReviewNumber	Number of user reviews in the category-week	18.633	11.822	0.000	70.000
AppNumber	Number of existing apps in each category in a given week	0.263	0.112	0.027	0.539
PlatformUpdate	Dummy (=1 Firefox update in this week; =0 otherwise)	0.030	0.169	0.000	1.000
AppReviewTime	Average number of days for app review in each category-week	6.525	1.196	5.015	8.590

exogenous shock, (ii) the app category–week combination as our unit of analysis, and (iii) two groups that receive the exogenous shock: the *Expansion* group of app categories (which had a new app release in a specific week) and the *Non-Expansion* group of app categories (as a comparison or baseline group that is otherwise equivalent to the *Expansion* group but did not have an new app release in that specific week). Drawing on this research design, our empirical analysis strategy is to evaluate the difference in *observed outcomes* (subsequent original new development and copycatting) between the *Expansion* group and the *Non-Expansion* group both before and after the *exogenous shock* (provisioning of SDK APIs).

Accordingly, we identified the *Expansion* group as app categories that had released at least one original new app in a specific week. The *Expansion* group is matched with the *Non-Expansion* group of app categories (as a baseline group that did not have any released new apps in the same week). The *Expansion* group has a greater potential to trigger subsequent new app development, and therefore is likely to expose stronger effects of APIs. First, new released apps can reveal some innovative ideas that inspire peer developers to develop similar or related apps, or even directly induce copycatting. Second, even if new released apps may not directly provide development ideas, they can signal market prospects of the corresponding app categories and attract more app suppliers to develop in these categories. Such a difference in the potential to trigger subsequent new app development can make the effects of APIs more salient on the *Expansion* group and allow us to better characterize these API effects.

We used propensity score matching (PSM) to match the *Expansion* group with the *Non-Expansion* group, so as to control for other unobservable confounding factors that may influence new app development. In PSM, we estimated propensity scores for all category–week combinations based on a set of selected covariates. In theory, propensity scores are the latent probabilities of receiving the treatment given the covariates (Stuart 2010). In our case, the *Expansion* group and the *Non-Expansion* group are matched based on their similar latent probabilities of releasing original new apps in given weeks. The covariates we used in estimating propensity scores are app category characteristics that are likely to influence new app releases, including the number of existing apps, the number of new original apps in the last week, the number of reviews, and the average app rating. We used nearest neighbor-matching technique to match each *Expansion* case with a closest *Non-Expansion* case based on predicted propensity score.

Propensity score matching results in 191 pairs of *Expansion* and *Non-Expansion* cases. Table C1 in online Appendix C shows that the covariate means for the *Expansion* group and the *Non-Expansion* group are comparable. Table C1 also reports the absolute standardized difference in means for each covariate, which is a commonly used measure of the balance of covariate distribution between two groups (Stuart 2010). The absolute standardized differences in means are all below the threshold of 0.25, suggesting a reasonable balance or matching between the two groups (Rubin 2001).

3.4. Difference-in-Differences Model Specifications

We estimate the differences between the pre-SDK period and the post-SDK period for the *Expansion* and *Non-Expansion* groups using a DID approach. The logic here is to compare the relative difference between the group of interest (the *Expansion* group) and the comparison group, both before and after the exogenous shock of a policy change. Doing so helps control for unobservable sources of heterogeneity across groups. In our case, the *Expansion* group and the *Non-Expansion* group differ in triggering subsequent new app development (either original new development or copycatting). If APIs facilitate new app development, we should observe that the

difference in increase of new app development between groups is more salient in the post-API period than in the pre-API period.

For each pair of *Expansion* and *Non-Expansion* groups, we considered the releases of new apps in subsequent four weeks following the *Expansion* week. We chose four weeks as our observation window because it may take time for some developers to respond to the market changes. We also varied length of observation windows in robustness analyses. Therefore, our main analysis at the category–week level contains 1528 (191 × 2 × 4) observations. Our model specifications are as follows:

$$\begin{aligned} OriginalNew_{it} = {} & \beta_0 + \beta_1 Expansion_i + \beta_2 PostSDK_t \\ & + \beta_3 Expansion_i \times PostSDK_t \\ & + \beta_4 MktPotential_{it} + \beta_5 MktConcen_{it} \\ & + \beta_6 MktMktAppComple_{it} \\ & + [ControlVars]_{it} + \varepsilon_{it} \end{aligned} \tag{1}$$

$$\begin{aligned} CopycatNew_{it} = {} & \beta_0 + \beta_1 Expansion_i + \beta_2 PostSDK_t \\ & + \beta_3 Expansion_i \times PostSDK_t \\ & + \beta_4 MktPotential_{it} + \beta_5 MktConcen_{it} \\ & + \beta_6 MktMktAppComple_{it} \\ & + [ControlVars]_i t + \varepsilon_{it} \end{aligned} \tag{2}$$

where i is the category index and t is the week index. *Expansion* is a dummy indicator with a value of 0 for the *Non-Expansion* group, and a value of 1 for the *Expansion* group. The coefficient of the interaction, that is, β_3, is the coefficient of interest. It reflects the effect of APIs on the *Expansion* group relative to the *Non-Expansion* group. To further examine how the impact of SDK APIs is influenced by market conditions and app complexity, we added three factors {*MktPotential*, *MktConcen*, *MktMktAppComple*} as a moderator, along with all lower-order interactions. The specifications are as follows:

$$\begin{aligned} OriginalNew_{it} = {} & \beta_0 + \beta_1 Expansion_i + \beta_2 PostSDK_t \\ & + \beta_3 Expansion_i \times PostSDK_t \\ & + \beta_4 MktPotential_{it} + \beta_5 MktConcen_{it} \\ & + \beta_6 MktAppComplex_{it} + \beta_7 Expansion_{it} \\ & \times Moderator_{it} + \beta_8 PostSDK_{it} \\ & \times Moderator_{it} + \beta_9 Expansion_{it} \\ & \times PostSDK_{it} \times Moderator_{it} \\ & + [ControlVars]_{it} + \varepsilon_{it} \end{aligned} \tag{3}$$

$$\begin{aligned} CopycatNew_{it} = {} & \beta_0 + \beta_1 Expansion_i + \beta_2 PostSDK_t \\ & + \beta_3 Expansion_i \times PostSDK_t \\ & + \beta_4 MktPotential_{it} + \beta_5 MktConcen_{it} \\ & + \beta_6 MktAppComple_{it} + \beta_7 Expansion_{it} \\ & \times Moderator_{it} + \beta_8 PostSDK_{it} \\ & \times Moderator_{it} + \beta_9 Expansion_{it} \\ & \times PostSDK_{it} \times Moderator_{it} \\ & + [ControlVars]_{it} + \varepsilon_{it} \end{aligned} \tag{4}$$

where $Moderator_{it}$ = {$MktPotential_{it}$, $MktConcen_{it}$, $MktAppComple_{it}$}.

We also included a set of dummies in all models to control for the "relative weeks" of observations. Relative weeks are the number of weeks elapsed since the corresponding occurrence of *Expansion*. Table 2 shows the summary statistics for original new apps and copycat apps before and after the release of SDK APIs.

4. Results

4.1. Hypotheses Testing

Tables 3 and 4 report our regression results. In Tables 3 and 4, the coefficients of *Expansion* are positive and significant in some models, suggesting that new app releases may trigger subsequent original new development and copycatting. This is consistent with our expectation. The coefficients of *PostAPIs* are all positive and significant, suggesting that APIs may facilitate both original new development of apps and app copycatting. These results provide some support to H1a and H1b. However, it is worth remarking that caution should be used in this case to draw conclusions about causal effects. This is mainly because certain unobservable factors along the time dimension, for example, the trend of app market growth, may also contribute to the increases in original new apps and copycats, and confound the effects of SDK APIs.

Table 2 Summary Statistics for Original New Development and App Copycatting

	Pre-SDK APIs Mean (SE)	Post-SDK APIs Mean (SE)
Number of original new apps (log)		
Expansion group	1.397 (0.013)	1.647 (0.017)
Non-Expansion group	1.351 (0.012)	1.534 (0.017)
Number of copycat new apps (log)		
Expansion group	1.229 (0.010)	1.337 (0.013)
Non-Expansion group	1.181 (0.008)	1.327 (0.012)

Note: $^*p < 0.10$; $^{**}p < 0.05$; $^{***}p < 0.01$; standard errors reported in parentheses.

Therefore, we also focus on the interaction of *Expansion* with *PostAPIs*.

The coefficients of the interaction between *Expansion* and *PostAPIs* in the models for original new development are positive and significant, and those in the models for app copycatting are negative and significant. Therefore, compared to the *Non-Expansion* group, the *Expansion* group exhibits a greater increase in original new development and a smaller increase in app copycatting after the release of SDK APIs. In particular, the release of SDK APIs resulted in about 7.2% greater increase in original new apps (Table 3, column 1) in the *Expansion* group than in the *Non-Expansion* group, and about 3.4% less increase in copycat apps (Table 4, column 1). These results provide evidence in support of the causal influence of SDK APIs on original new development of apps, further supporting H1a. The results also support H1b. Although app copycatting increased on average in the post-API period, the relative use of original new development in comparison to copycatting is clearly increased in the post-API period as well. A plausible reason is that app suppliers, especially those experienced ones, were motivated to lean more toward using original new development in response to market expansion, which inevitably counteracted the enhancing effect of APIs on copycatting.

Column (2) in both Tables 3 and 4 shows a positive and significant coefficient for the three-way interaction (*Expansion* × *PostAPIs* × *MktPotential*). These results suggest that, with a greater market potential for apps, the effect of SDK APIs becomes more prominent in enhancing original new development of apps. However, by reducing the negative effect of *Expansion* × *PostAPIs*, market potential reinforced SDK APIs in enhancing app copycatting as well. Hence, H2a and H2b are supported. A plausible explanation is that greater market potential provides more opportunities for both original developers and imitators to benefit from API-based development.

Column (3) in Table 3 shows that in the model for original new development, the coefficient of the three-way interaction (*Expansion* × *PostAPIs* × *MktConcen*) is not significant. The suggestion is that the enhancing effect of SDK APIs on original new development of apps is not influenced by market concentration. Therefore, H3a is not supported. In contrast, column (3) in Table 4 shows that the coefficient of this three-way interaction is positive and significant in the model for copycat apps. Therefore, market concentration reduces the negative effect of *Expansion* × *PostAPIs* and essentially enhances the overall effect of SDK APIs on app copycatting. H3b is supported. A plausible explanation is that market concentration leaves market followers with fewer choices other than copycatting.

Column (4) in Table 3 shows that the coefficient of the three-way interaction term (*Expansion* × *PostAPIs* × *MktAppComplex*) is positive and significant in the model for original new app

Table 3 Analysis on Number of Original New Apps (*OriginalNew*)

	(1)	(2)	(3)	(4)
Expansion	0.015 (0.011)	0.062* (0.036)	0.092*** (0.021)	0.019 (0.021)
PostAPIs	0.200*** (0.029)	0.207*** (0.037)	0.194*** (0.052)	0.208*** (0.039)
Expansion × *PostAPIs*	0.072*** (0.020)	0.068 (0.045)	0.042 (0.058)	0.051 (0.042)
MktPotential	0.291*** (0.073)	0.693*** (0.166)	0.279*** (0.079)	0.360*** (0.074)
MktConcen	0.057 (0.042)	0.048 (0.040)	0.135*** (0.044)	0.031 (0.055)
MktAppComplex	−0.019 (0.013)	−0.024 (0.018)	−0.021 (0.014)	−0.035*** (0.012)
Expansion × *MktPotential*		−0.328* (0.200)		
PostAPIs × *MktPotential*		−0.189 (0.197)		
Expansion × *PostAPIs* × *MktPotential*		0.368** (0.136)		
Expansion × *MktConcen*			−0.204*** (0.044)	
PostAPIs × *MktConcen*			−0.002 (0.053)	
Expansion × *PostAPIs* × *MktConcen*			0.079 (0.112)	
Expansion × *MktAppComplex*				−0.025 (0.024)
PostAPIs × *MktAppComplex*				0.019 (0.064)
Expansion × *PostAPIs* × *MktAppComplex*				0.146*** (0.043)
CatGrowth	0.519*** (0.194)	0.393 (0.341)	0.692*** (0.159)	0.564 (0.580)
AppRating	−0.043 (0.035)	−0.045 (0.037)	−0.042 (0.035)	−0.052 (0.055)
ReviewNumber	0.001 (0.001)	0.001 (0.001)	0.001 (0.001)	0.001 (0.001)
AppNumber	0.479*** (0.065)	0.448*** (0.068)	0.488*** (0.062)	0.543*** (0.116)
PlatformUpdate	0.188*** (0.036)	0.190*** (0.040)	0.192*** (0.038)	0.220*** (0.053)
AppReviewTime	0.004 (0.007)	0.015 (0.012)	0.005 (0.008)	0.004 (0.010)
Dummies of Relative Week	Included	Included	Included	Included
N	1,528	1,528	1,528	1,528
Adjusted R^2	0.192	0.205	0.194	0.197

Note: *$p < 0.10$; **$p < 0.05$; ***$p < 0.01$; standard errors reported in parentheses.

Table 4 Analysis on Number of New Copycat Apps (*CopycatNew*)

	(1)	(2)	(3)	(4)
Expansion	0.043*** (0.010)	0.027 (0.026)	0.079*** (0.023)	0.036*** (0.014)
PostAPIs	0.124*** (0.015)	0.123*** (0.038)	0.102*** (0.027)	0.122*** (0.022)
Expansion × *PostAPIs*	−0.034** (0.015)	−0.062** (0.028)	−0.063* (0.036)	−0.045* (0.026)
MktPotential	0.054 (0.043)	0.056*** (0.017)	0.081 (0.059)	0.049 (0.046)
MktConcen	−0.065 (0.043)	−0.073 (0.055)	−0.098** (0.036)	−0.066 (0.043)
MktAppComplex	0.074*** (0.015)	0.071*** (0.014)	0.061*** (0.013)	0.050*** (0.012)
Expansion × *MktPotential*		−0.143 (0.141)		
PostAPIs × *MktPotential*		−0.246 (0.203)		
Expansion × *PostAPIs* × *MktPotential*		0.326** (0.135)		
Expansion × *MktConcen*			−0.152*** (0.052)	
PostAPIs × *MktConcen*			−0.160*** (0.054)	
Expansion × *PostAPIs* × *MktConcen*			0.151*** (0.048)	
Expansion × *MktAppComplex*				0.031 (0.029)
PostAPIs × *MktAppComplex*				0.013 (0.038)
Expansion × *PostAPIs* × *MktAppComplex*				0.059 (0.069)
CatGrowth	−0.238 (0.224)	−0.251 (0.232)	−0.293 (0.258)	−0.290 (0.282)
AppRating	−0.029 (0.028)	−0.035 (0.028)	−0.038 (0.031)	−0.029 (0.035)
ReviewNumber	0.002** (0.001)	0.002** (0.001)	0.001 (0.001)	0.002** (0.001)
AppNumber	0.406*** (0.047)	0.396*** (0.046)	0.478*** (0.074)	0.401*** (0.077)
PlatformUpdate	−0.025 (0.019)	−0.024 (0.018)	−0.015 (0.029)	−0.023 (0.038)
AppReviewTime	−0.015*** (0.004)	−0.009** (0.004)	−0.014*** (0.005)	−0.015** (0.006)
Dummies of Relative Week	Included	Included	Included	Included
N	1,528	1,528	1,528	1,528
Adjusted R^2	0.165	0.171	0.172	0.165

Note: *$p < 0.10$; **$p < 0.05$; ***$p < 0.01$; standard errors reported in parentheses.

development. This result suggests that the effect of SDK APIs in enhancing original new app development is stronger in categories with greater market-level app complexity, which supports H4a. A plausible explanation is that the facilitating mechanism of APIs is more crucial in markets where apps are more difficult to develop. Column (4) in Table 4 shows that the coefficient of the three-way interaction term is not significant in the model for copycats. In other words, the effect of APIs in reducing copycats is not influenced by market-level app complexity. Therefore, H4b is not supported. A potential explanation is that the intrinsic complexity of apps can counteract the effect of APIs in simplifying original app creation through the mix-and-match approach.

4.2. Additional Insights

We conducted additional analyses to complement main findings. First, we divided the 14 categories of apps broadly into two groups: apps that extend Firefox functionalities and apps that offer distinct new functionalities.[1] For the two types of apps, we did a sub-group analysis to rerun the model. The results, as summarized in online Appendix D, indicate that the effect of platform APIs in increasing original new development is relatively stronger, and its effect in increasing app copycatting is relatively weaker for app categories extending functionalities of Firefox. For the influence of platform APIs on original new development, the moderating effects of market potential and market-level app complexity are relatively stronger for app categories offering new functionalities. For the influence of platform APIs on app copycatting, the moderating effect of market potential and market concentration is relatively stronger for app categories extending functionalities of Firefox. These results provide some initial evidences that the effect of platform APIs on original new development of apps and app copycatting may differ for apps, depending on the nature of apps.

Second, we conducted sensitivity analysis to check for hidden bias—the presence of unobserved factors that may affect the assignment into the *Expansion* group and the development outcomes simultaneously (Rosenbaum 2002). Essentially, this involves investigating whether some unobservable factors influence the treatment effect. For this assessment, we conducted a sensitivity analysis based on the bounds recommended by Rosenbaum (2002). The results are summarized in online Appendix E. As shown in Table E1, Δ is a measure of the degree of departure from the analysis that is free of hidden bias (Becker and Caliendo 2007), where $\Delta = 1$ suggests the hidden-bias-free analysis. The *p*-values are used for testing whether the conclusions of the study are altered by the corresponding departure of Δ caused by hidden bias. The results suggest that even with a departure of as much as $\Delta = 3$, the identified difference between the *Expansion* group and the *Non-Expansion* group (i.e., which is equivalent to a treatment effect) still

exists (i.e., the altering is rejected at $p < 0.0001$). Therefore, our matching method is robust to the presence of unobservables, and hidden bias is not a serious concern in our case.

4.3. Robustness Checks

We conducted several checks to verify the robustness of our results with alternative measures and model specifications. To conserve space, the details of these robustness checks and related results are provided in online appendices. First, we employed an alternative window of three months for the pre-APIs period and the post-APIs period to examine the impact of SDK APIs (in online Appendix F). Second, we adopted an alternative threshold of similarity (0.95) for copycat detection (in online Appendix G). Third, we used the average number of code lines as an alternative measure of *MktAppComplex* to examine the effect of market-level app complexity (online Appendix H). Fourth, we reran the models using the ratio of original new apps to total apps (*OriginalNew/AppNumber*) as an alternative measure of original new development, and the ratio of new copycat apps to total apps (*CopycatNew/AppNumber*) as an alternative measure of app copycatting (online Appendix I). Fifth, we reran the models using normalized measures of original new apps and app copycatting for detrending (online Appendix J). Sixth, we used an alternative model specification of seemingly unrelated regression (SUR) to take into account the potential correlation between error terms of the two equations for original new apps and copycat apps (online Appendix K). Seventh, we used an alternative matching approach. Specifically, we conducted a 1-to-2 matching using the two-nearest neighbors with replacement to match each treated observation with 2 controlled observations, and used the enlarged matched sample to rerun the models (online Appendix L). Finally, we conducted two robustness checks using alternative model specifications to take into account the potential overlap between new app releases as the treatment and as the development outcomes (online Appendix M). The results of all of these robustness checks are qualitatively consistent with the main results, supporting the robustness of our main findings.

5. Discussion

5.1. Theoretical Implications

This research has several theoretical implications that contribute to multiple streams of literature. First, while the operations management literature has recognized the value of platform principles and modularity for NPD (Jiao et al. 2007, Jose and Tollenaere 2005), the focus has been largely on how individual producers employ platform principles to organize internal NPD and how platform architecture is employed for recombining components to develop product variety. Our study contributes to the OM literature by clarifying that platform principles have important implications for the governance of not only internal NPD, but also digital NPD in an ecosystem context. Platform APIs, which implement the principles of modularity and standardization across organizational boundaries, can serve as an effective design mechanism to coordinate digital product innovation by third-party suppliers. Using platform APIs to attract a large number of third-party NPD suppliers, organizations can go beyond coordinating NPD activities in supply chains to building open innovation ecosystems to enable emergence. This adds to understanding of how firms can design and govern their supply networks for innovation emergence as opposed to control (Choi et al. 2001). Moreover, the NPD based on software platforms and supported by APIs not only produces digital outputs, but also dramatically transforms the coordination and operation of the new development of physical product systems (Svahn et al. 2017).

Second, while the literature has focused on how individual producers can address the tension between innovation and imitation in relation to other competing producers in a supply chain, this study extends by considering how this tension can be addressed in a platform ecosystem. Our study thus integrated the concepts of product modularity along with innovation and imitation behavior (Ethiraj and Levinthal 2004, Ethiraj et al. 2008, Pil and Cohen 2006) in the context of innovation ecosystems. The important implication is that designing a modular platform architecture based on APIs can influence third-party suppliers to assess the trade-off between pursuing innovation and imitation in favor of innovation strategies. This insight extends the understanding of how the tension between imitation and innovation can be managed at the level of the individual producer in a supply chain to the level of a platform ecosystem. In addition, by adopting a granular measurement approach to distinguish original new development from copycatting, our study also adds to the literature on platform ecosystems that has largely focused on aggregate innovation outputs from third-party suppliers (e.g., Boudreau 2012, Song et al. 2018).

Third, the results pertaining to the contingent effect of market-level app complexity add to our understanding of operations management, specifically the influences of product complexity on the determinants of NPD performance (Peng et al. 2014, Vickery et al. 2016). Interestingly, focusing on *product* modularity as a coordination mechanism in the context of an internal product system, past work has found that product

complexity *dampens* the contribution of product modularity to overall new product performance (Vickery et al. 2016). In contrast to this finding, focusing on *platform* modularity as a coordination mechanism to facilitate third-party suppliers' development in the context of an open innovation ecosystem, we found that the effect of APIs on original new development of apps is *strengthened* when market-level app complexity is high. Our finding illuminates that modularizing the platform's resources through open-access APIs can facilitate third-party suppliers to better cope with more complex NPD tasks.

5.2. Practical Implications

Our study has two main practical implications. First, for platform operators, our findings suggest that the facilitating effect on original app development can be material enough to motivate the shift of development focus from imitation to original development. Platform operators may, therefore, consider investing more in the standardization of their API systems (e.g., simplifying calling methods, enhancing the loose coupling between functions, and consolidating low-level functions for high-level app development, etc.) to make it feasible for developers to allocate more attention and effort to creativity in third-party app development. In addition, platform operators may leverage market conditions to enhance the benefits of their API strategies. For example, they can use cross-side network effects by using user-side demand information to better motivate app suppliers to employ APIs to achieve innovative app development. They may also attract more diverse developers to limit market concentration to bolster original app development. Platform operators may also use other mechanisms (e.g., rewarding systems, app reviews) along with APIs to constrain copycatting in these market conditions.

Second, for app developers, our study suggests that they should focus more on original new development of apps relative to copycatting when adopting platform APIs, as these can help them realize their innovative ideas of development. Our study also suggests that third-party developers should, in general, be less concerned about the issue of copycatting when platforms support their ecosystems with more standardized APIs. The competition based on original development contribution, rather than on copycatting, is more likely to prevail with further advances in platform APIs.

5.3. Limitations and Future Research

Our study has some limitations and important implications for future research. First, although we focused on general platform concepts that can apply to other contexts of operations and new product development, our empirical investigation is limited to a single software platform, specifically Firefox. Future research may consider verifying and extending our ideas to other contexts of platform-based development and operations. Second, our empirical study focused on the release of one type of APIs (SDK APIs). The effect of platform APIs on third-party development behavior may depend on numerous platform functions underlying different types of platform APIs. Future research may examine different types of APIs with different platform functions to expand our findings. Furthermore, while our empirical study focused on a single platform, future research may replicate this study using multiple platforms. Examining original development and copycatting from a platform competition perspective and exploring the use of APIs to achieve competitive advantages can also be fruitful avenues.

6. Conclusion

Our study has important implications for both NPD in operations management and innovation in technology management. By empirically examining the app development ecosystem of a leading web browser platform, our study delivers a key message that platform APIs, which implement platform principles (e.g., modularization and standardization), have the potential to foster original new development of digital products by third-party app suppliers and address the issue of copycatting. Market conditions—specifically, market potential, market concentration, and technological complexity—can shape the influence of APIs on third-party suppliers' digital NPD behavior. Platform operators can leverage APIs in conjunction with market conditions to enhance digital NPD in platform ecosystems.

Acknowledgment

The authors are grateful to Subodha Kumar, the Senior Editor, and two referees for their constructive comments that have significantly improved the presentation and content of this study. The research of the corresponding author, Peijian Song, was supported by the National Natural Science Foundation of China under Grant #71472083. The research of the fourth author, Cheng Zhang, was supported by the National Natural Science Foundation of China under Grant # 71871065, and Grant # 91846302.

Note

[1]Apps extending Firefox functionalities include those in the category of appearance, alerts and updates, bookmark management, download management, language support, privacy and security, and tabs, while apps offering distinct

new functionalities include those in the categories of feeds/news/blogging, games & entertainment, photos/music/videos, shopping, social & communication, and web development.

References

Acemoglu, D., J. Linn. 2004. Market size in innovation: Theory and evidence from the pharmaceutical industry. *Q. J. Econ.* **119**(3): 1049–1090.

Baldwin, C., J. Woodard. 2009. The architecture of platforms: A unified view. A. Gawer, ed. *Platforms, Markets and Innovation*. Edward Elgar, Cheltenham, UK, 19–44.

Becker, S. O., M. Caliendo. 2007. Sensitivity analysis for average treatment effect. *Stata J.* **7**(1): 71–83.

Benzell, S. G., G. Lagarda, M. Van Alstyne. 2017. The impact of APIs on firm performance. Working paper, Boston University.

Bessen, J., E. Maskin. 2009. Sequential innovation, patents and imitation. *Rand J. Econ.* **40**(4): 611–635.

Bhargava, H. K., B. C. Kim, D. Sun. 2013. Commercialization of platform technologies: Launch timing and versioning strategy. *Prod. Oper. Manag.* **22**(6): 1374–1388.

Bikhchandani, S., D. Hirshleifer, I. Welch. 1998. Learning from the behavior of others: Conformity, fads, and informational cascades. *J. Econ. Perspect.* **12**(3): 151–170.

Billington, C., R. Davidson. 2013. Leveraging open innovation using intermediary networks. *Prod. Oper. Manag.* **22**(6): 1464–1477.

Bin-Habtoor, A. S., M. A. Zaher. 2012. A survey on plagiarism detection systems. *Int. J. Comput. Theory Eng.* **4**(2): 185–188.

Bockstedt, J., C. Druehl, A. Mishra. 2016. Heterogeneous submission behavior and its implications for success in innovation contests with public submissions. *Prod. Oper. Manag.* **25**(7): 1157–1176.

Boudreau, K. J. 2012. Let a thousand flowers bloom? An early look at large numbers of software app developers and patterns of innovation. *Organ. Sci.* **23**(5): 1409–1427.

Burrows, S., S. M. Tahaghoghi, J. Zobel. 2007. Efficient plagiarism detection for large code repositories. *Software Pract. Exper.* **37**(2): 151–175.

Choi, T. Y., K. L. Dooley, M. Rungtusanatham. 2001. Supply networks and complex adaptive systems: Control versus emergence. *Prod. Oper. Manag.* **19**(3): 351–366.

Chu, J., P. Manchanda. 2016. Quantifying cross-network effects in online C2C platforms. *Market. Sci.* **35**(6): 870–893.

Cosma, G., M. Joy. 2012. An approach to source-code plagiarism detection and investigation using latent semantic analysis. *IEEE Trans. Comput.* **61**(3): 379–394.

Curry, B., K. D. George. 1983. Industrial concentration: A survey. *J. Ind. Econ.* **31**(3): 203–255.

Danese, P., R. Filippini. 2013. Direct and mediated effects of product modularity on development time and product performance. *IEEE Trans. Eng. Manage.* **60**(2): 260–271.

Domis, O. 2016. "Uber's APIs: Giving developers the keys to innovation," Apigee.com. Available at https://apigee.com/about/blog/api-technology/ubers-apis-giving-developers-keys-innovation (accessed date August 2, 2016).

Ethiraj, S. K., D. Levinthal. 2004. Modularity and innovation in complex systems. *Management Sci.* **50**(2): 159–173.

Ethiraj, S. K., D. Levinthal, R. R. Roy. 2008. The dual role of modularity: Innovation and imitation. *Management Sci.* **54**(5): 939–955.

Evans, D. S., A. Hagiu, R. Schmalensee. 2006. *Invisible Engines: How Software Platforms Drive Innovation and Transform Industries*. MIT Press, Cambridge, MA.

Fabrizio, K. R., L. G. Thomas. 2012. The impact of local demand on innovation in a global industry. *Strateg. Manag. J.* **33**(1): 42–64.

Gawer, A. 2014. Bridging differing perspectives on technological platforms: Toward an integrative framework. *Res. Policy* **43**(7): 1239–1249.

Guha, S., S. Kumar. 2018. Emergence of big data research in operations management, information systems, and healthcare: Past contributions and future roadmap. *Prod. Oper. Manag.* **27**(9): 1724–1735.

Hao, L., H. Guo, R. F. Easley. 2017. A mobile platform's in-app advertising contract under agency pricing for app sales. *Prod. Oper. Manag.* **26**(2): 189–202.

Henderson, R. M., K. B. Clark. 1990. Architectural innovation: The reconfiguration of existing product technologies and the failure of established firms. *Adm. Sci. Q.* **35**(1): 9–30.

Iyer, B., M. Subramaniam, 2015. The strategic value of APIs. *Harvard Business Review*, available at https://hbr.org/2015/01/the-strategic-value-of-apis (accessed date July 1, 2015).

Jacobson, D., G. Brail, D. Woods. 2012. *APIs: A Strategy Guide*. O'Reilly, Sebastopol, CA.

Jiao, J., T. W. Simpson, Z. Siddique. 2007. Product family design and platform-based product development: A state-of-the-art review. *J. Intell. Manuf.* **18**(1): 5–29.

Jose, A., M. Tollenaere. 2005. Modular and platform methods for product family design: Literature analysis. *J. Intell. Manuf.* **16**(3): 371–390.

Kearney, M. S., P. B. Levine. 2015. Media influences on social outcomes: The impact of MTV's "16 and Pregnant" on TeenChildbearing. *Am. Econ. Rev.* **105**(12): 3597–3632.

Kontostathis, A., W. M. Pottenger. 2006. A framework for understanding latent semantic indexing (LSI) performance. *Inf. Process. Manage.* **42**(1): 56–73.

Krishnan, V., C. H. Loch. 2005. A retrospective look at production and operations management articles on new product development. *Prod. Oper. Manag.* **14**(4): 433–441.

Lau, A. K. W., R. C. M. Yam, E. Tang. 2011. The impact of product modularity on new product performance: Mediation by product innovativeness. *J. Prod. Innov. Manag.* **28**(2): 270–284.

Lee, H. L., G. Schmidt. 2017. Using value chains to enhance innovation. *Prod. Oper. Manag.* **26**(4): 617–632.

Lieberman, M. B., S. Asaba. 2006. Why do firms imitate each other. *Acad. Manag. Rev.* **31**(2): 366–385.

Lukashenko, R., V. Graudina, J. Grundspenkis. 2007. Computer-based plagiarism detection methods and tools: An overview. Proceedings of the 2007 International Conference on Computer Systems and Technologies, no. 40, Bulgaria.

Lyytinen, K., G. M. Rose. 2003. The disruptive nature of information technology innovations: The case of internet computing in systems development organizations. *MIS Q.* **27**(4): 557–596.

Moody, G. D., D. F. Galletta. 2015. Lost in cyberspace: The impact of information scent and time constraints on stress, performance, and attitudes online. *J. Manage. Inf. Syst.* **32**(1): 192–224.

Mozgovoy, M. 2006. Desktop tools for offline plagiarism detection in computer grograms. *Inform. Educ.* **5**(1): 97–112.

Peng, D. X., G. R. Heim, D. N. Mallick. 2014. Collaborative product development: The effect of project complexity on the use of information technology tools and new product development practices. *Prod. Oper. Manag.* **23**(8): 1421–1438.

Pil, F. K., S. K. Cohen. 2006. Modularity: Implications for imitation, innovation, and sustained advantage. *Acad. Manag. Rev.* **31**(4): 995–1011.

Pun, H., G. D. DeYong. 2017. Competing with copycats when customers are strategic. *Manuf. Serv. Oper. Manag.* **19**(3): 403–418.

Rai, A., L. M. Maruping, V. Venkatesh. 2009. Offshore information systems project success: The role of social embeddedness and cultural characteristics. *MIS Q.* **33**(3): 617–641.

Rivkin, J. 2000. Imitation of complex strategies. *Management Sci.* **46**(6): 824–845.

Rosenbaum, P. R. 2002. *Observational Studies*. Springer, New York.

Rubin, D. B. 2001. Using propensity scores to help design observational studies: Application to the tobacco litigation. *Health Serv. Outcomes Res. Method.* **2**(3): 169–188.

Sanchez, R., J. T. Mahoney. 1996. Modularity, flexibility, and knowledge management in product and organization design. *Strateg. Manag. J.* **17**: 63–76.

Scherer, F. M. 1992. Schumpeter and plausible capitalism. *J. Econ. Lit.* **30**(3): 1416–1433.

Schilling, M. A. 2000. Towards a general modular systems theory and its application to interfirm product modularity. *Acad. Manag. Rev.* **25**(2): 312–324.

Shadish, W., T. Cook, D. Campbell. 2002. *Experimental and Quasi-Experimental Designs for Generalized Causal Inference*. Houghton Mifflin, Boston, MA.

Sidorova, A., N. Evangelopoulos, J. S. Valacich, T. Ramakrishnan. 2008. Uncovering the intellectual core of the information systems discipline. *MIS Q.* **32**(3): 467–482.

Song, P., L. Xue, A. Rai, C. Zhang. 2018. The ecosystem of software platform: A study of asymmetric cross-side network effects and platform governance. *MIS Q.* **42**(1): 121–142.

StatCounter. 2017. StatCounter global stats—Desktop browser. Available at http://gs.statcounter.com/browser-market-share/desktop/worldwide (accessed date January 2018).

Stuart, E. A. 2010. Matching methods for causal inference: A review and a look forward. *Stat. Sci.* **25**(1): 1–21.

Svahn, F., L. Mathiassen, R. Lindgren, G. C. Kane. 2017. Mastering the digital innovation challenge. *Sloan Manage. Rev.* **58**(3): 14–16.

Tiwana, A. 2014. *Platform Ecosystems: Aligning Architecture, Governance, and Strategy*. Elsevier Inc., Waltham, MA.

Turner, S. F., W. Mitchell, R. A. Bettis. 2010. Responding to rivals and complements: How market concentration shapes generational product innovation strategy. *Organ. Sci.* **21**(4): 854–872.

Ulrich, K. T. 1995. The role of product architecture in the manufacturing firms. *Res. Policy* **24**(3): 419–440.

Vickery, S. K., X. Koufteros, C. Dröge, R. Calantone. 2016. Product modularity, process modularity, and new product introduction performance: Does complexity matter. *Prod. Oper. Manag.* **25**(4): 751–770.

Von Hippel, E., R. Katz. 2002. Shifting innovation to users via toolkits. *Management Sci.* **48**(7): 821–834.

Wang, Q., B. Li, P. V. Singh. 2018. Copycats vs. original mobile apps: A machine learning copycat-detection method and empirical analysis. *Inf. Syst. Res.* **29**(2): 273–291.

Yoo, Y., R. J. Boland, K. Lyytinen, A. Majchrzak. 2012. Organizing for innovation in the digitized world. *Organ. Sci.* **23**(5): 1398–1408.

Zahra, S. A., S. R. Das. 1993. Innovation strategy and financial performance in manufacturing companies: An empirical study. *Prod. Oper. Manag.* **2**(1): 15–37.

Zhang, L. L. 2015. A literature review on multitype platforming and framework for future research. *Int. J. Prod. Econ.* **168**: 1–12.

Supporting Information

Additional supporting information may be found online in the Supporting Information section at the end of the article.

Appendix A. Firfox's APIs and Assessing Code File Similarity Using SVD.

Appendix B. Correlation Matrix for Variables.

Appendix C. Descriptive Statistics of Covariates after Propensity Score Matching.

Appendix D. Sub-Group Analysis.

Appendix E. Sensitivity Analysis for Checking Hidden Bias.

Appendix F. Using Three Months as Alternative Window for Pre-APIs and Post-APIs Period.

Appendix G. Choose 0.95 as Alternative Threshold of Similarity for Copycat Detection.

Appendix H. Using Number of Code Lines as Measure of Market-Level App Complexity.

Appendix I. Using Ratio of Original New Apps and Ratio of New Copycat Apps as Measurement of Innovation and Imitation.

Appendix J. Using Normalized Measures of Original New Apps and Copycat Apps as Measurement of Innovation and Imitation.

Appendix K. Using Seemingly Unrelated Regression (SUR) to Account the Potential Correlation between the Error Terms of Two Equations.

Appendix L. Using the Two-Nearest Neighbors with Replacement as Alternative Matching Approach.

Appendix M. Analysis Addressing Overlapped Effects of New App Releases.